

Projektarbeit

Certified AI Engineer

Entwicklung und Vergleich von
Machine-Learning-Modellen zur
Vorhersage des Wettmarktes „Both
Teams To Score“ in der englischen
Premier League

verfasst im Rahmen der EN ISO / IEC 17024
Zertifizierungsprüfung von

Jennifer Dapprich

17.07.2026

Eidesstattliche Erklärung

Hiermit versichere ich an Eides statt, dass ich die vorliegende Projektarbeit eigenständig und ohne Mitwirkung Dritter angefertigt habe. Jegliche verwendeten Quellen und Hilfsmittel wurden entsprechend als solche gekennzeichnet. Dies betrifft insbesondere auch jene Teile der Arbeit, die mittels künstlicher Intelligenz (z.B. ChatGPT) generiert und/oder bearbeitet wurden. Die Kenntlichmachung erfolgt nach Maßgabe der geltenden Digethic-Zitationsregeln.

Es ist mir ferner bewusst, dass die erbrachte Eigenleistung bei Anfertigung der Projektarbeit min. 90% betragen muss. Sollten Teile der Arbeit durch künstliche Intelligenz generiert oder bearbeitet worden sein, darf dies entsprechend 10% des gesamten Projektarbeitsumfangs nicht überschreiten.

Ich stimme ausdrücklich der Verwendung eines Plagiatsprogramms zu, anhand dessen unter anderem die Verwendung von künstlicher Intelligenz überprüft werden kann. Ich bin mir dessen bewusst, dass ein Verstoß gegen diese Erklärung ein Nichtbestehen der Prüfung nach sich zieht und zum Verlust des Prüfungsanspruchs führen kann.

München, 17.07.2026

Ort, Datum, Unterschrift

A handwritten signature in black ink, appearing to be 'Fengler', written over a horizontal line.

Inhalt

1	Einleitung.....	1
2	Analyse.....	1
2.1	Ausgangssituation.....	1
2.2	Ziel des Projekts.....	2
2.3	Datenquelle.....	2
2.4	Zielvariable BTTS.....	3
3	Umsetzung.....	3
3.1	Projektumgebung.....	3
3.2	Datenaufbereitung.....	4
3.3	Feature Engineering.....	5
3.3.1	Durchschnittlich erzielte Tore.....	5
3.3.2	Durchschnittlich kassierte Gegentore.....	6
3.3.3	Aktuelle Form der Mannschaft.....	6
3.3.4	Erweiterte Features.....	6
3.4	Entwicklung der Machine-Learning-Modelle.....	7
3.4.1	Logistische Regression.....	7
3.4.2	Neuronales Netz.....	7
3.5	Modellbewertung.....	8
3.6	Ergebnisse und Modellvergleich.....	9
4	Reflexion.....	11
5	Resümee und Ausblick.....	12
6	Einsatz von KI-Werkzeugen.....	13
	Literaturverzeichnis.....	13
	Abbildungsverzeichnis.....	14
	Tabellenverzeichnis.....	15
	Anhang.....	15
	Anhang A: Quellcode des Projekts.....	15
	Anhang B: Übersicht der verwendeten Features.....	16
	Anhang C: Hyperparameter des neuronalen Netzes.....	16

1 Einleitung

Fußball gehört zu den beliebtesten Sportarten der Welt und wird zunehmend durch Datenanalysen unterstützt. Nach jedem Spiel stehen zahlreiche Statistiken wie Tore, Schüsse, Ballbesitz oder Fouls zur Verfügung. Vereine, Analysten und Medien nutzen diese Daten, um Spiele auszuwerten und Entscheidungen zu unterstützen. Gleichzeitig entwickelt sich der Bereich der Künstlichen Intelligenz stetig weiter. Insbesondere Machine Learning bietet die Möglichkeit, aus großen Datenmengen Muster zu erkennen und daraus Vorhersagen für zukünftige Ereignisse abzuleiten (Géron, 2022, S. 4 ff.).

Da ich mich seit vielen Jahren für Fußball interessiere, wollte ich herausfinden, ob sich Methoden des Machine Learnings dazu eignen, den Wettmarkt „Both Teams To Score“ (BTTS) vorherzusagen. Dabei geht es um die Frage, ob beide Mannschaften in einem Spiel mindestens ein Tor erzielen. Obwohl diese Fragestellung zunächst einfach erscheint, wird das Ergebnis von vielen verschiedenen Faktoren beeinflusst. Dazu gehören unter anderem die aktuelle Form einer Mannschaft, ihre Offensiv- und Defensivleistung sowie die Ergebnisse der vergangenen Spiele.

Ziel dieser Projektarbeit ist die Entwicklung eines künstlichen neuronalen Netzes, das auf Grundlage historischer Spieldaten vorhersagt, ob in einem Premier-League-Spiel beide Mannschaften mindestens ein Tor erzielen. Hierfür werden zunächst historische Spieldaten mehrerer Spielzeiten analysiert und aufbereitet. Anschließend werden geeignete Merkmale (Features) entwickelt, die die aktuelle Leistungsfähigkeit der Mannschaften möglichst gut beschreiben.

Um die Leistungsfähigkeit verschiedener Verfahren einordnen zu können, werden zwei unterschiedliche Modellklassen entwickelt. Zunächst entsteht eine logistische Regression als Baseline-Modell. Anschließend wird ein künstliches neuronales Netz entwickelt und trainiert. Die Qualität der Modelle wird anhand der Kennzahlen Accuracy, Precision, Recall und F1-Score bewertet und verglichen.

Als methodischer Rahmen dient das Vorgehensmodell CRISP-DM, das den Ablauf eines Data-Mining-Projekts in die Phasen Business Understanding, Data Understanding, Data Preparation, Modeling, Evaluation und Deployment gliedert (Chapman et al., 2000, S. 9 ff.). Abschließend werden die Ergebnisse kritisch reflektiert und mögliche Verbesserungen für zukünftige Arbeiten aufgezeigt.

2 Analyse

2.1 Ausgangssituation

Durch die zunehmende Digitalisierung stehen im Profifußball heute große Mengen an Daten zur Verfügung. Nach jedem Spiel werden zahlreiche Statistiken wie Tore, Schüsse, Ballbesitz oder Fouls

erfasst. Diese Daten bilden die Grundlage für Analysen und ermöglichen den Einsatz von Machine-Learning-Verfahren zur Vorhersage zukünftiger Ereignisse.

Machine-Learning-Modelle können große Datenmengen auswerten und Zusammenhänge erkennen, die mit klassischen statistischen Methoden nur schwer sichtbar werden. Dadurch eignen sie sich besonders für Vorhersagen, bei denen viele verschiedene Einflussfaktoren berücksichtigt werden müssen (Goodfellow, Bengio & Courville, 2016, S. 96 ff.).

Ein praxisnaher Anwendungsbereich ist der Wettmarkt „Both Teams To Score“ (BTTS). Hier wird vorhergesagt, ob beide Mannschaften mindestens ein Tor erzielen. Da diese Entscheidung von vielen Faktoren abhängt, eignet sich der Wettmarkt gut, um den Einsatz von Machine Learning in einem überschaubaren Projekt zu untersuchen.

2.2 Ziel des Projekts

Ziel dieser Projektarbeit ist die Entwicklung eines Machine-Learning-Modells zur Vorhersage von BTTS-Ereignissen auf Grundlage historischer Spieldaten der englischen Premier League. Dabei steht nicht nur die Entwicklung eines möglichst guten Vorhersagemodells im Mittelpunkt, sondern auch das praktische Verständnis des gesamten Machine-Learning-Prozesses, von der Datenaufbereitung bis zur Bewertung der Modelle.

Dafür wurden zunächst mehrere Spielzeiten zusammengeführt und aufbereitet. Anschließend wurden verschiedene Merkmale entwickelt, die die aktuelle Form sowie die offensive und defensive Leistung der Mannschaften beschreiben. Auf Basis dieser Daten wurden zwei verschiedene Modellklassen entwickelt und miteinander verglichen.

Als Baseline dient eine logistische Regression. Anschließend wurde ein künstliches neuronales Netz entwickelt, um zu untersuchen, ob ein komplexeres Modell bessere Vorhersagen liefert. Ein Modell gilt in dieser Arbeit dann als erfolgreich, wenn es die Baseline messbar übertrifft.

2.3 Datenquelle

Als Datenquelle wurde die frei zugängliche Plattform football-data.co.uk verwendet (Football-Data.co.uk, 2026). Sie stellt historische Spieldaten verschiedener europäischer Fußballligen zur Verfügung und eignet sich aufgrund ihrer Datenqualität und ihres Umfangs gut für Machine-Learning-Projekte.

Für diese Projektarbeit wurden fünf Spielzeiten der englischen Premier League verwendet. Nach dem Zusammenführen der einzelnen CSV-Dateien standen rund 1.900 Spiele mit umfangreichen Spielstatistiken zur Verfügung. Der Datensatz enthielt unter anderem Informationen zu Toren, Schüssen, Schüssen auf das Tor, Fouls, Karten und Wettquoten. Die Daten wurden im CSV-Format heruntergeladen und anschließend mit der Python-Bibliothek Pandas eingelesen und verarbeitet

(McKinney, 2010, S. 56 ff.). Dadurch konnten die einzelnen Spielzeiten zu einem gemeinsamen Datensatz zusammengeführt werden.

Die Premier League wurde ausgewählt, da sie über einen vollständigen und konsistenten Datensatz verfügt und gleichzeitig eine der bekanntesten Fußballligen der Welt ist.

2.4 Zielvariable BTTS

Die Zielvariable dieser Projektarbeit ist „Both Teams To Score“ (BTTS). Für jedes Spiel wurde überprüft, ob sowohl die Heim- als auch die Auswärtsmannschaft mindestens ein Tor erzielt hat.

Erzielen beide Mannschaften mindestens ein Tor, erhält das Spiel den Wert 1. Bleibt mindestens eine Mannschaft ohne Tor, wird der Wert 0 vergeben. Es handelt sich damit um ein binäres Klassifikationsproblem.

Nach der Erstellung der Zielvariable zeigte sich, dass etwa 55 % der Spiele ein BTTS-Ereignis darstellten. Dadurch entstand ein relativ ausgewogener Datensatz, der sich gut für die Entwicklung eines Klassifikationsmodells eignet. Ein ausgeprägtes Klassenungleichgewicht, das gesonderte Gegenmaßnahmen erfordern würde, lag somit nicht vor.

3 Umsetzung

3.1 Projektumgebung

Für die Umsetzung des Projekts wurde die Programmiersprache Python verwendet (Python Software Foundation, 2026). Python bietet zahlreiche Bibliotheken für Datenanalyse und Machine Learning und eignet sich daher besonders gut für die Entwicklung eines solchen Projekts. Für die Verwaltung der Abhängigkeiten wurde eine virtuelle Umgebung (venv) eingesetzt.

Als Entwicklungsumgebung kam Visual Studio Code zum Einsatz. Um die verschiedenen Arbeitsschritte übersichtlich zu strukturieren, wurde das Projekt in mehrere Jupyter Notebooks aufgeteilt. Jedes Notebook übernahm dabei einen klar abgegrenzten Aufgabenbereich, von der Datenanalyse über das Feature Engineering bis hin zum Training und der Bewertung der Modelle. Dadurch konnten die einzelnen Arbeitsschritte, vom Einlesen der Daten bis zur Auswertung der Modelle, klar voneinander getrennt werden.

Für die Versionsverwaltung wurde Git genutzt. Zusätzlich wurde das Projekt in einem GitHub-Repository gespeichert, sodass Änderungen jederzeit nachvollziehbar waren und der aktuelle Projektstand gesichert werden konnte.

Für die Umsetzung wurden die folgenden Python-Bibliotheken eingesetzt:

Tabelle 1: Im Projekt eingesetzte Python-Bibliotheken und ihr jeweiliger Verwendungszweck.

Bibliothek	Verwendung
Pandas	Einlesen und Bearbeiten der Spieldaten
NumPy	Mathematische Berechnungen
Scikit-learn	Logistische Regression, Datenaufteilung und Standardisierung
TensorFlow / Keras	Entwicklung des neuronalen Netzes
Matplotlib	Visualisierung der Ergebnisse
Joblib	Speichern und Laden des Modells

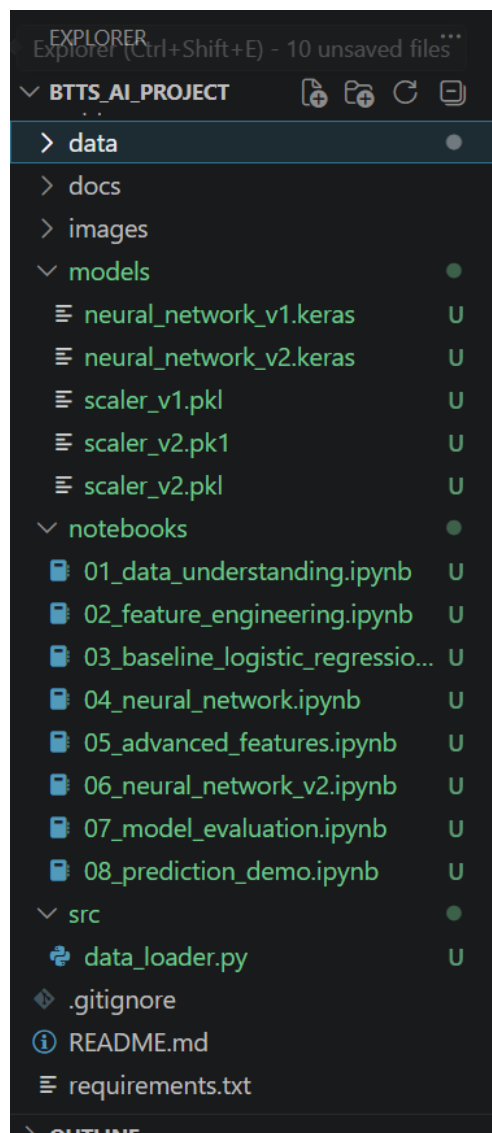


Abbildung 1: Projektstruktur in Visual Studio Code.

3.2 Datenaufbereitung

Zu Beginn des Projekts wurden die historischen Spieldaten der englischen Premier League von football-data.co.uk heruntergeladen. Da die Daten für jede Saison als einzelne CSV-Datei bereitgestellt werden,

mussten zunächst alle Dateien eingelesen und zu einem gemeinsamen Datensatz zusammengeführt werden.

```
dfs = []  
  
for file in files:  
    df = pd.read_csv(os.path.join(DATA_PATH, file))  
    df["Season"] = file.replace(".csv", "")  
    dfs.append(df)  
  
matches = pd.concat(dfs, ignore_index=True)
```

Abbildung 2: Zusammenführen der einzelnen CSV-Dateien zu einem gemeinsamen Datensatz.

Nach dem Zusammenführen wurden die Daten auf Vollständigkeit überprüft und für die weitere Verarbeitung vorbereitet. Anschließend wurde die Zielvariable BTTS erstellt. Dafür wurden die Tore der Heim- und Auswärtsmannschaft ausgewertet. Erzielten beide Mannschaften mindestens ein Tor, erhielt das Spiel den Wert 1, andernfalls den Wert 0.

```
matches["BTTS"] = ((matches["FTHG"] > 0) & (matches["FTAG"] > 0)).astype(int)
```

Abbildung 3: Erstellung der Zielvariable BTTS aus den Toren beider Mannschaften.

Im nächsten Schritt wurden nur die für das Projekt relevanten Merkmale ausgewählt. Dadurch entstand eine übersichtliche Datenbasis, die als Grundlage für das Feature Engineering und das Training der Modelle diente.

3.3 Feature Engineering

Die ursprünglichen Spieldaten enthielten bereits zahlreiche Informationen, beispielsweise zu Toren, Schüssen oder Fouls. Für die Vorhersage von BTTS-Ereignissen wurden diese Daten jedoch nicht direkt verwendet. Stattdessen wurden zusätzliche Merkmale (Features) entwickelt, die die aktuelle Form sowie die offensive und defensive Leistung der Mannschaften besser beschreiben.

Dabei wurde bewusst entschieden, nur die letzten fünf Spiele einer Mannschaft zu berücksichtigen. Dadurch fließen aktuelle Leistungen stärker in das Modell ein, während ältere Ergebnisse weniger Einfluss haben. Alle Features wurden ausschließlich aus Spielen berechnet, die vor der jeweiligen Partie stattgefunden haben. Damit wird verhindert, dass Informationen aus der Zukunft in das Modell gelangen und ein Datenleck entsteht.

3.3.1 Durchschnittlich erzielte Tore

Als erstes Feature wurde die durchschnittliche Anzahl der erzielten Tore der letzten fünf Spiele berechnet. Dabei wurden Heim- und Auswärtsmannschaften getrennt betrachtet. So entstanden die Merkmale HomeGoalsLast5 und AwayGoalsLast5.

Diese Features sollen die aktuelle Offensivstärke einer Mannschaft beschreiben. Es wurde angenommen, dass Mannschaften mit einer höheren durchschnittlichen Toranzahl in den vergangenen Spielen auch im nächsten Spiel mit einer höheren Wahrscheinlichkeit mindestens ein Tor erzielen.

```
matches["HomeGoalsLast5"] = (  
    matches.groupby("HomeTeam")["FTHG"]  
    .transform(lambda x: x.shift(1).rolling(5, min_periods=1).mean())  
)
```

Abbildung 4: Berechnung der durchschnittlich erzielten Tore der letzten fünf Spiele.

Die Berechnung erfolgt rollierend über die jeweils letzten fünf Spiele einer Mannschaft. Dadurch werden ausschließlich Informationen verwendet, die bereits vor dem aktuellen Spiel bekannt waren.

3.3.2 Durchschnittlich kassierte Gegentore

Neben der Offensivleistung wurde auch die Defensivleistung berücksichtigt. Dafür wurden die durchschnittlich kassierten Gegentore der letzten fünf Spiele berechnet.

Die daraus entstandenen Features HomeConcededLast5 und AwayConcededLast5 geben Hinweise darauf, wie stabil die Defensive einer Mannschaft zuletzt war. Es wurde angenommen, dass eine Mannschaft, die zuletzt viele Gegentore kassiert hat, auch im nächsten Spiel mit einer höheren Wahrscheinlichkeit mindestens ein Tor zulässt. Damit steigt zugleich die Wahrscheinlichkeit für ein BTTS-Ereignis.

3.3.3 Aktuelle Form der Mannschaft

Zusätzlich wurde die Form der Mannschaften über die letzten fünf Spiele betrachtet. Für jedes Spiel wurden Punkte vergeben:

1. Sieg = 3 Punkte
2. Unentschieden = 1 Punkt
3. Niederlage = 0 Punkte

Aus diesen Punkten wurde anschließend ein Durchschnitt berechnet. Die entstandenen Features HomeFormLast5 und AwayFormLast5 sollen die aktuelle Leistungsfähigkeit der Mannschaften unabhängig von der Anzahl der erzielten Tore beschreiben.

```
matches["HomeFormLast5"] = (  
    matches.groupby("HomeTeam")["HomePoints"]  
    .transform(lambda x: x.shift(1).rolling(5, min_periods=1).mean())  
)
```

Abbildung 5: Berechnung des Features HomeFormLast5 aus den Punkten der letzten fünf Spiele.

3.3.4 Erweiterte Features

Nachdem das erste neuronale Netz trainiert und bewertet worden war, wurden weitere Features entwickelt, um zu untersuchen, ob sich die Vorhersageleistung verbessern lässt. Dazu gehörten:

4. die durchschnittliche Anzahl der Schüsse auf das Tor der letzten fünf Spiele,
5. die durchschnittliche BTTS-Quote der letzten fünf Spiele.

Diese zusätzlichen Merkmale wurden in einem zweiten neuronalen Netz verwendet. Der spätere Vergleich zeigte jedoch, dass die Erweiterung des Datensatzes in diesem Fall nicht zu einer besseren Modellleistung führte. Dies verdeutlicht, dass eine größere Anzahl an Features nicht automatisch zu besseren Vorhersagen führt. Entscheidend ist vielmehr, wie aussagekräftig die einzelnen Merkmale sind.

3.4 Entwicklung der Machine-Learning-Modelle

Nachdem die Daten aufbereitet und die Features erstellt worden waren, konnten die Modelle entwickelt werden. Ziel war es, die Wahrscheinlichkeit vorherzusagen, dass in einem Premier-League-Spiel beide Mannschaften mindestens ein Tor erzielen.

Um die Ergebnisse besser bewerten zu können, wurden zwei unterschiedliche Modellklassen eingesetzt. Zunächst wurde eine logistische Regression als Baseline-Modell entwickelt. Anschließend wurde ein künstliches neuronales Netz erstellt und mit denselben Daten trainiert. Durch diesen Vergleich konnte untersucht werden, ob ein komplexeres Modell tatsächlich bessere Vorhersagen liefert.

3.4.1 Logistische Regression

Die logistische Regression diente als erstes Modell und bildete die Vergleichsgrundlage für das weitere Projekt. Dieses Verfahren eignet sich besonders für binäre Klassifikationsprobleme, da die Zielvariable nur zwei mögliche Werte annehmen kann, also $BTTS = 1$ oder $BTTS = 0$ (Géron, 2022, S. 148 ff.).

Vor dem Training wurden die Daten in Trainings- und Testdaten aufgeteilt. Zusätzlich wurden die Eingabedaten standardisiert, damit alle Merkmale denselben Wertebereich besitzen. Anschließend wurde das Modell mit der Bibliothek Scikit-learn trainiert und anhand der Testdaten bewertet (Pedregosa et al., 2011, S. 2825 ff.).

```
model = LogisticRegression()  
  
model.fit(X_train, y_train)
```

Abbildung 6: Training der logistischen Regression als Baseline-Modell.

Die Abbildung zeigt die Initialisierung und das Training der logistischen Regression. Nach der Standardisierung der Eingabedaten wurde das Modell auf den Trainingsdaten trainiert und anschließend mit den Testdaten bewertet.

3.4.2 Neuronales Netz

Nach der Entwicklung des Baseline-Modells wurde ein künstliches neuronales Netz erstellt. Ziel war es zu untersuchen, ob ein komplexeres Modell die Zusammenhänge in den Daten besser erkennen kann.

Das neuronale Netz bestand aus einer Eingabeschicht, zwei versteckten Schichten mit ReLU-Aktivierungsfunktion sowie einer Ausgabeschicht mit einer Sigmoid-Aktivierungsfunktion. Die Sigmoid-Funktion gibt einen Wert zwischen 0 und 1 aus und eignet sich daher für die Ausgabe einer Wahrscheinlichkeit. Zusätzlich wurde eine Dropout-Schicht eingesetzt, um das Risiko des Overfittings zu verringern (Srivastava et al., 2014, S. 1929 ff.). Die gewählte Architektur wurde bewusst einfach gehalten, um zunächst eine solide Vergleichsbasis zur logistischen Regression zu schaffen und das Risiko eines zu komplexen Modells zu vermeiden.

Das Modell wurde mit dem Adam-Optimierer und der Verlustfunktion Binary Crossentropy über 50 Epochen trainiert (Kingma & Ba, 2015). Die Umsetzung erfolgte mit TensorFlow und der Keras-Schnittstelle (Abadi et al., 2016). Während des Trainings wurden sowohl die Entwicklung des Trainings- und Validierungsfehlers als auch die Trainings- und Validierungsgenauigkeit beobachtet, um mögliche Anzeichen von Overfitting frühzeitig zu erkennen.

```
model = Sequential([
    Input(shape=(6,)),
    Dense(32, activation="relu"),
    Dropout(0.2),
    Dense(16, activation="relu"),
    Dense(1, activation="sigmoid")
])

model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

model.summary()
```

Abbildung 7: Architektur des entwickelten neuronalen Netzes.

Die Modellübersicht verdeutlicht den Aufbau des neuronalen Netzes mit zwei versteckten Schichten und einer Sigmoid-Ausgabeschicht. Insgesamt besitzt das Modell 769 trainierbare Parameter.

3.5 Modellbewertung

Nach dem Training wurden beide Modelle anhand der Testdaten bewertet. Zur Beurteilung der Modellleistung wurden die Kennzahlen Accuracy, Precision, Recall und F1-Score verwendet.

Die Accuracy gibt an, wie viele Spiele insgesamt korrekt vorhergesagt wurden. Precision beschreibt den Anteil der tatsächlich richtigen BTTS-Vorhersagen unter allen positiven Vorhersagen. Recall zeigt, wie viele der tatsächlichen BTTS-Ereignisse vom Modell erkannt wurden. Der F1-Score kombiniert Precision und Recall zu einer gemeinsamen Kennzahl und eignet sich besonders für den Vergleich verschiedener Klassifikationsmodelle (Géron, 2022, S. 92 ff.). Zusätzlich wurde die Confusion Matrix

ausgewertet, um die Verteilung richtiger und falscher Klassifikationen besser nachvollziehen zu können. Aufgrund des Fokus dieser Arbeit werden jedoch ausschließlich die wichtigsten Bewertungskennzahlen dargestellt.

Für das neuronale Netz wurde zusätzlich die Entwicklung der Loss-Werte während des Trainings betrachtet. Dadurch konnte überprüft werden, ob das Modell sinnvoll gelernt hat oder Anzeichen von Overfitting zeigte.

Der Verlauf in Abbildung 8 zeigt ein typisches Bild von Overfitting: Der Trainingsfehler sinkt über die 50 Epochen kontinuierlich, während der Validierungsfehler bereits nach wenigen Epochen wieder ansteigt. Das Modell passt sich also zunehmend an die Trainingsdaten an, ohne dabei besser zu generalisieren. Trotz der eingesetzten Dropout-Schicht reichte die Regularisierung nicht aus, um diesen Effekt vollständig zu verhindern. Ein früheres Beenden des Trainings (Early Stopping) nach etwa fünf bis zehn Epochen wäre hier die naheliegende Gegenmaßnahme und sollte in einer Weiterentwicklung des Modells berücksichtigt werden.

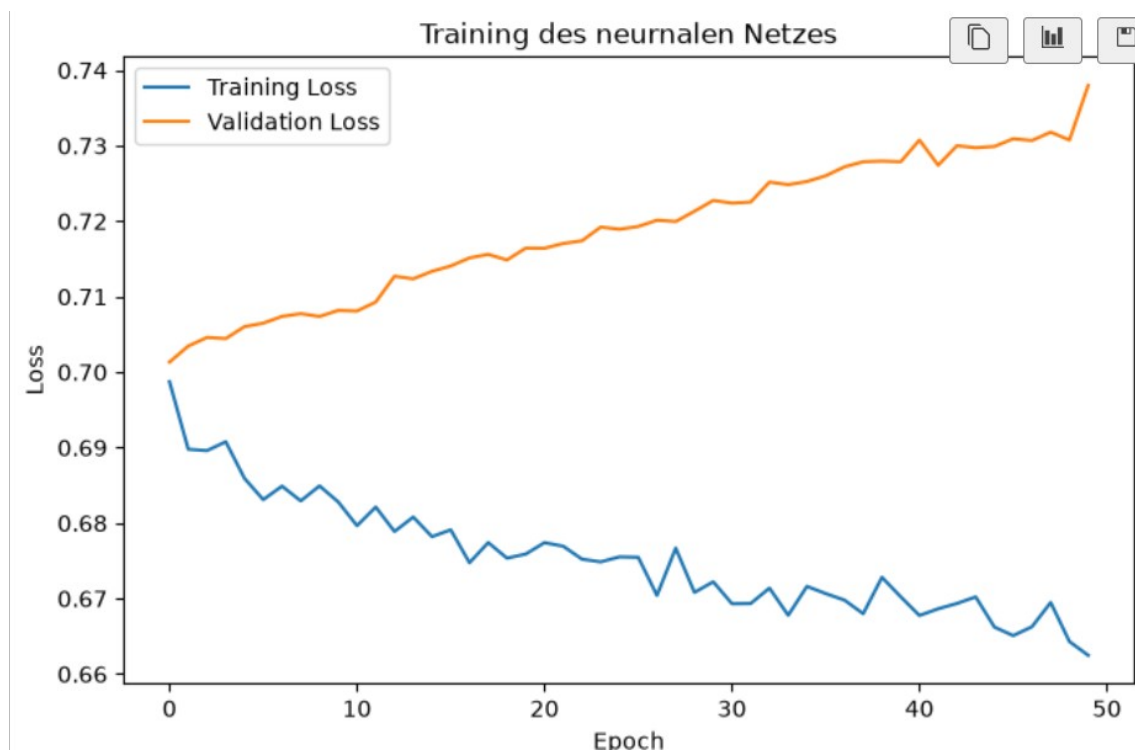


Abbildung 8: Verlauf des Trainings- und Validierungsfehlers (Loss) des neuronalen Netzes.

3.6 Ergebnisse und Modellvergleich

Nach dem Training der Modelle wurden die Ergebnisse anhand der Testdaten ausgewertet. Dadurch konnte die Vorhersagequalität der verschiedenen Modelle miteinander verglichen werden.

Die logistische Regression diente als Baseline-Modell. Sie erreichte eine Accuracy von 54,8 % und erkannte einen Großteil der tatsächlichen BTTS-Ereignisse. Gleichzeitig zeigte sich jedoch, dass die Gesamtleistung des Modells begrenzt war.

Das erste neuronale Netz erzielte die besten Ergebnisse im gesamten Projekt. Mit einer Accuracy von 59,1 % sowie dem höchsten F1-Score zeigte dieses Modell insgesamt die ausgewogenste Leistung. Dies deutet darauf hin, dass das neuronale Netz komplexere Zusammenhänge in den Daten besser erfassen konnte als die logistische Regression. Trotz der insgesamt besseren Ergebnisse fiel die Verbesserung gegenüber der logistischen Regression geringer aus als zunächst erwartet.

Im nächsten Schritt wurde das Modell durch zusätzliche Features erweitert. Dazu gehörten die durchschnittliche Anzahl der Schüsse auf das Tor sowie die BTTS-Quote der letzten fünf Spiele. Ziel war es zu untersuchen, ob diese zusätzlichen Informationen die Vorhersage verbessern.

Die Ergebnisse zeigten jedoch, dass das zweite neuronale Netz keine Leistungssteigerung erreichte. Die Accuracy sank leicht auf 54,0 %, während der Recall zwar geringfügig höher ausfiel, Precision und F1-Score jedoch schlechter waren als beim ersten Modell.

Tabelle 2: Vergleich der Bewertungskennzahlen der drei entwickelten Modelle auf den Testdaten.

Modell	Accuracy	Precision	Recall	F1-Score
Logistische Regression	0,548	0,572	0,904	0,702
Neuronales Netz V1	0,591	0,597	0,927	0,726
Neuronales Netz V2	0,540	0,541	0,929	0,684

Zur besseren Veranschaulichung werden die Ergebnisse der drei Modelle zusätzlich grafisch dargestellt. Die folgende Abbildung zeigt den Vergleich der vier Bewertungskennzahlen.

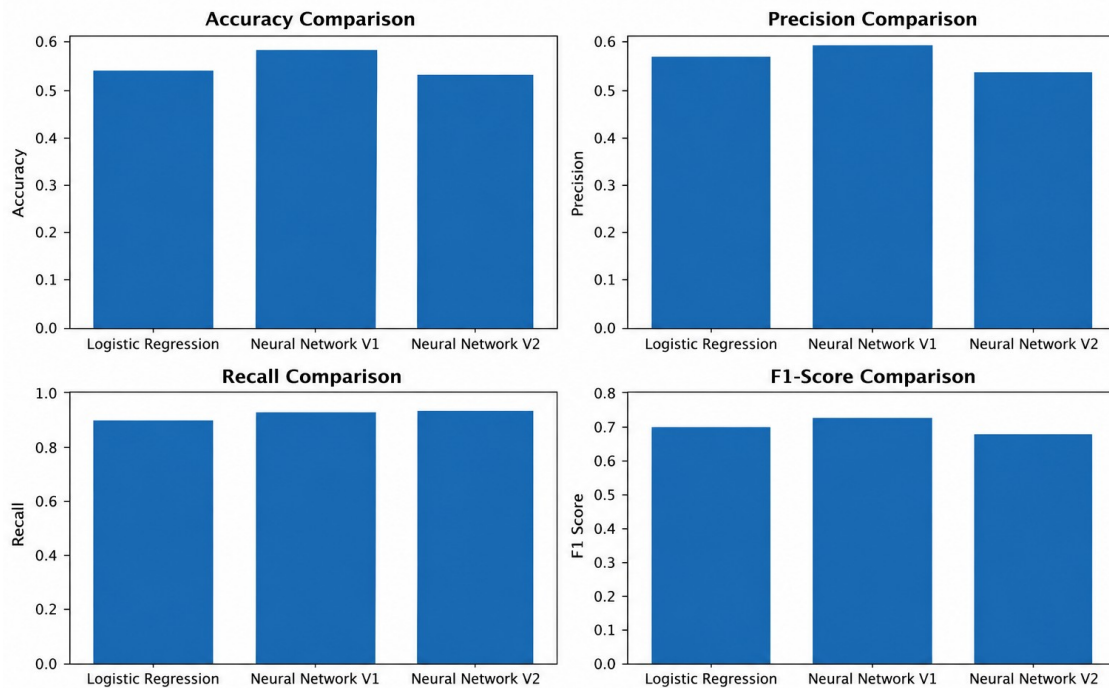


Abbildung 9: Vergleich der Accuracy-, Precision-, Recall- und F1-Score-Werte der entwickelten Machine-Learning-Modelle.

Abbildung 9: Grafischer Vergleich von Accuracy, Precision, Recall und F1-Score der drei Modelle.

Alle drei Modelle erreichten einen hohen Recall. Das zweite neuronale Netz erzielte den höchsten Wert und erkannte den größten Anteil tatsächlicher BTTS-Ereignisse. Gleichzeitig ging dies jedoch zulasten der Precision und des F1-Scores. Der hohe Recall bei gleichzeitig niedriger Precision zeigt, dass die Modelle dazu neigen, sehr häufig auf BTTS zu tippen. Insgesamt zeigte der Vergleich, dass zusätzliche Features nicht automatisch zu besseren Ergebnissen führen. Entscheidend ist vielmehr, wie aussagekräftig und relevant die verwendeten Merkmale für die Vorhersage sind.

4 Reflexion

Während der Umsetzung des Projekts konnten viele praktische Erfahrungen im Bereich Machine Learning gesammelt werden. Besonders deutlich wurde, dass die Qualität der Daten und der entwickelten Features einen großen Einfluss auf die Leistungsfähigkeit eines Modells haben.

Zu Beginn des Projekts wurde eine logistische Regression als Baseline-Modell entwickelt. Dadurch entstand eine Vergleichsgrundlage, anhand derer die Ergebnisse des neuronalen Netzes bewertet werden konnten. Das erste neuronale Netz erzielte insgesamt die besten Ergebnisse und erreichte sowohl die höchste Accuracy als auch den besten F1-Score. Dies zeigt, dass ein neuronales Netz die Zusammenhänge der verwendeten Daten besser erfassen konnte als die logistische Regression.

Im weiteren Verlauf wurden zusätzliche Features entwickelt, darunter die durchschnittliche Anzahl der Schüsse auf das Tor sowie die BTTS-Quote der letzten fünf Spiele. Ziel war es, die Vorhersageleistung weiter zu verbessern. Die Ergebnisse zeigten jedoch, dass das zweite neuronale Netz trotz der

zusätzlichen Informationen keine besseren Ergebnisse erzielte. Daraus wurde deutlich, dass eine größere Anzahl an Merkmalen nicht automatisch zu einer höheren Modellqualität führt. Wichtiger ist die Auswahl aussagekräftiger Features.

Kritisch anzumerken ist, dass die erreichte Accuracy von 59,1 % nur begrenzt über der Trefferquote liegt, die sich durch die konsequente Vorhersage der Mehrheitsklasse ergeben würde. Fußball ist stark zufallsbehaftet, weshalb die theoretische Vorhersagegrenze niedrig ist. Ein ehrlicher Vergleich mit den Quoten der Buchmacher, die als anspruchsvolle Baseline gelten, wurde im Rahmen dieser Arbeit nicht durchgeführt und bleibt eine offene Fragestellung.

Während der Entwicklung traten außerdem verschiedene technische Herausforderungen auf. Dazu gehörten fehlende Werte bei den ersten Spielen einer Saison, die korrekte Berechnung der Features auf Basis der letzten fünf Spiele sowie die Einrichtung und Nutzung von TensorFlow innerhalb der Entwicklungsumgebung. Diese Probleme konnten Schritt für Schritt gelöst werden und führten zu einem besseren Verständnis des gesamten Machine-Learning-Prozesses.

Insgesamt hat das Projekt gezeigt, dass die Entwicklung eines Machine-Learning-Modells nicht nur aus dem Training eines Algorithmus besteht. Einen wesentlichen Anteil am Projekterfolg hatten die Datenaufbereitung, das Feature Engineering und die anschließende Auswertung der Ergebnisse. Gerade diese Arbeitsschritte erwiesen sich als entscheidend für die Qualität der Vorhersagen. Durch das Projekt habe ich nicht nur den Umgang mit Python und verschiedenen Machine-Learning-Bibliotheken gelernt, sondern auch ein besseres Verständnis für den gesamten Entwicklungsprozess gewonnen. Rückblickend würde ich künftig mehr Zeit in die Entwicklung und Auswahl geeigneter Features investieren, da diese im Projekt den größten Einfluss auf die Modelleistung hatten.

5 Resümee und Ausblick

Ziel dieser Projektarbeit war die Entwicklung und der Vergleich verschiedener Machine-Learning-Modelle zur Vorhersage des Wettmarktes „Both Teams To Score“ auf Grundlage historischer Spieldaten der englischen Premier League.

Dafür wurden zunächst historische Spieldaten mehrerer Spielzeiten aufbereitet und um zusätzliche Features erweitert. Anschließend wurden eine logistische Regression sowie zwei neuronale Netze entwickelt und miteinander verglichen. Die Bewertung erfolgte anhand der Kennzahlen Accuracy, Precision, Recall und F1-Score.

Die Ergebnisse zeigten, dass das erste neuronale Netz insgesamt die beste Leistung erzielte. Es erreichte die höchste Accuracy sowie den besten F1-Score und lieferte damit die ausgewogensten Vorhersagen. Die logistische Regression eignete sich als Baseline-Modell und bot eine gute

Vergleichsgrundlage, wurde jedoch hinsichtlich der Gesamtleistung vom neuronalen Netz übertroffen. Das zweite neuronale Netz mit zusätzlichen Features konnte die Ergebnisse dagegen nicht weiter verbessern. Dies verdeutlicht, dass nicht die Anzahl der Merkmale entscheidend ist, sondern deren Aussagekraft.

Durch das Projekt konnte der vollständige Ablauf eines Machine-Learning-Projekts praktisch umgesetzt werden, von der Datenaufbereitung über das Feature Engineering bis hin zum Training und der Bewertung verschiedener Modelle. Dabei wurden wertvolle Erfahrungen im Umgang mit Python, Machine Learning und der Auswertung von Modellen gesammelt.

Für zukünftige Arbeiten könnte das Projekt durch zusätzliche Daten oder alternative Verfahren erweitert werden. Denkbar wäre die Einbindung weiterer Ligen, aussagekräftigerer Statistiken wie Expected Goals (xG) oder moderner Modelle wie Gradient Boosting und XGBoost. Außerdem könnte untersucht werden, ob sich durch eine systematische Optimierung der Hyperparameter, eine Kalibrierung der vorhergesagten Wahrscheinlichkeiten oder eine größere Datenbasis eine weitere Verbesserung der Vorhersagequalität erzielen lässt.

Zusammenfassend konnte gezeigt werden, dass Machine Learning grundsätzlich zur Vorhersage des Wettmarktes BTTS eingesetzt werden kann. Die Ergebnisse machen jedoch auch deutlich, dass die Qualität der Daten und der entwickelten Features einen entscheidenden Einfluss auf die Leistungsfähigkeit eines Modells haben und dass die erreichbare Vorhersagegüte in einer stark zufallsbehafteten Sportart wie Fußball begrenzt bleibt. Insgesamt hat das Projekt gezeigt, dass Machine Learning interessante Möglichkeiten für die Analyse von Fußballspielen bietet, gleichzeitig aber auch die Grenzen datenbasierter Vorhersagen im Sport deutlich werden.

6 Einsatz von KI-Werkzeugen

Für die sprachliche Formulierung einzelner Textpassagen sowie zur Unterstützung bei der Strukturierung der Projektarbeit wurde OpenAI ChatGPT (GPT-5.5) verwendet. Sämtliche Inhalte wurden fachlich geprüft, eigenständig überarbeitet und an das tatsächlich durchgeführte Projekt angepasst.

Literaturverzeichnis

- Abadi, M., Barham, P., Chen, J., Chen, Z., Davis, A., Dean, J., Devin, M., Ghemawat, S., Irving, G., Isard, M., Kudlur, M., Levenberg, J., Monga, R., Moore, S., Murray, D. G., Steiner, B., Tucker, P., Vasudevan, V., Warden, P., Wicke, M., Yu, Y. & Zheng, X. (2016). TensorFlow: A system for large-scale machine learning. In Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI '16) (S. 265–283). USENIX Association.
- Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., Shearer, C. & Wirth, R. (2000). CRISP-DM 1.0: Step-by-step data mining guide. SPSS Inc.

- Football-Data.co.uk (2026). England Football Results and Betting Odds Data. <https://www.football-data.co.uk/englandm.php> [Abgerufen am: 07.07.2026]
- Géron, A. (2022). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems (3. Aufl.). O'Reilly Media. ISBN 978-1-098-12597-4.
- Goodfellow, I., Bengio, Y. & Courville, A. (2016). Deep Learning. MIT Press. ISBN 978-0-262-03561-3.
- Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Río, J. F., Wiebe, M., Peterson, P., ... Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585(7825), 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95. <https://doi.org/10.1109/MCSE.2007.55>
- Kingma, D. P. & Ba, J. (2015). Adam: A method for stochastic optimization. In *Proceedings of the 3rd International Conference on Learning Representations (ICLR 2015)*. <https://arxiv.org/abs/1412.6980>
- McKinney, W. (2010). Data structures for statistical computing in Python. In *Proceedings of the 9th Python in Science Conference (SciPy 2010)* (S. 56–61). <https://doi.org/10.25080/Majora-92bf1922-00a>
- Pandas Development Team (2026). pandas documentation. <https://pandas.pydata.org/docs/> [Abgerufen am: TT.MM.2026]
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M. & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Python Software Foundation (2026). Python 3 documentation. <https://docs.python.org/3/> [Abgerufen am: TT.MM.2026]
- Scikit-learn Developers (2026). scikit-learn User Guide. https://scikit-learn.org/stable/user_guide.html [Abgerufen am: TT.MM.2026]
- Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I. & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1), 1929–1958.
- TensorFlow Developers (2026). TensorFlow API Documentation. https://www.tensorflow.org/api_docs [Abgerufen am: TT.MM.2026]

Abbildungsverzeichnis

Abbildung 1: Projektstruktur in Visual Studio Code.

Abbildung 2: Zusammenführen der einzelnen CSV-Dateien zu einem gemeinsamen Datensatz.

Abbildung 3: Erstellung der Zielvariable BTTS aus den Toren beider Mannschaften.

Abbildung 4: Berechnung der durchschnittlich erzielten Tore der letzten fünf Spiele.

Abbildung 5: Berechnung des Features HomeFormLast5 aus den Punkten der letzten fünf Spiele.

Abbildung 6: Training der logistischen Regression als Baseline-Modell.

Abbildung 7: Architektur des entwickelten neuronalen Netzes.

Abbildung 8: Verlauf des Trainings- und Validierungsfehlers (Loss) des neuronalen Netzes.

Abbildung 9: Grafischer Vergleich von Accuracy, Precision, Recall und F1-Score der drei Modelle.

Tabellenverzeichnis

Tabelle 1: Im Projekt eingesetzte Python-Bibliotheken und ihr jeweiliger Verwendungszweck.

Tabelle 2: Vergleich der Bewertungskennzahlen der drei entwickelten Modelle auf den Testdaten.

Tabelle 3: Übersicht aller im Projekt entwickelten Features.

Tabelle 4: Verwendete Hyperparameter des neuronalen Netzes.

Anhang

Anhang A: Quellcode des Projekts

Der vollständige Quellcode des Projekts einschließlich aller Jupyter Notebooks ist im GitHub-

Repository hinterlegt: <https://github.com/jenniferdapprich-bot/football-btts-ai.git>

Anhang B: Übersicht der verwendeten Features

Tabelle 3: Übersicht aller im Projekt entwickelten Features.

Feature	Beschreibung	Modell
HomeGoalsLast5	Durchschnittlich erzielte Tore der Heimmannschaft in den letzten fünf Spielen	V1, V2
AwayGoalsLast5	Durchschnittlich erzielte Tore der Auswärtsmannschaft in den letzten fünf Spielen	V1, V2
HomeConcededLast5	Durchschnittlich kassierte Gegentore der Heimmannschaft in den letzten fünf Spielen	V1, V2
AwayConcededLast5	Durchschnittlich kassierte Gegentore der Auswärtsmannschaft in den letzten fünf Spielen	V1, V2
HomeFormLast5	Durchschnittliche Punktausbeute der Heimmannschaft in den letzten fünf Spielen	V1, V2
AwayFormLast5	Durchschnittliche Punktausbeute der Auswärtsmannschaft in den letzten fünf Spielen	V1, V2
HomeShotsOnTargetLast5	Durchschnittliche Schüsse auf das Tor der Heimmannschaft in den letzten fünf Spielen	nur V2
AwayShotsOnTargetLast5	Durchschnittliche Schüsse auf das Tor der Auswärtsmannschaft in den letzten fünf Spielen	nur V2
HomeBTTSRateLast5	Anteil der BTTS-Ereignisse in den letzten fünf Spielen der Heimmannschaft	nur V2
AwayBTTSRateLast5	Anteil der BTTS-Ereignisse in den letzten fünf Spielen der Auswärtsmannschaft	nur V2

Anhang C: Hyperparameter des neuronalen Netzes

Tabelle 4: Verwendete Hyperparameter des neuronalen Netzes.

Parameter	Wert
Versteckte Schichten	2 (ReLU-Aktivierung)
Ausgabeschicht	1 Neuron (Sigmoid-Aktivierung)
Regularisierung	Dropout
Optimierer	Adam
Verlustfunktion	Binary Crossentropy
Epochen	50
Vorverarbeitung	Standardisierung der Eingabemerkmale

